



Department of CSE

Mini Project Report - Task 2

“Malware Family Classification Using CNN Baselines and a Patch-Based GCN on the MaleBin Dataset”

Course Code: ICE 478/CSE475

Section: 2

Track: Track 2 - GCN (Image -> Graph)

Submitted to: Dr. Raihan Ul Islam, Associate Professor, Department of Computer Science and Engineering.

Submitted by: Group-1

Student ID	Student Name
2022-3-50-006	Md. Mustafidur Rahman
2022-3-50-008	Amana Akter Arfa
2022-3-50-025	Rakiba Alam

1. Introduction

This Task 2 report follows Track 2 of the CSE475/ICE478 project: converting each malware image into a graph and evaluating a Graph Convolutional Network (GCN) against representative CNN baselines. The MaleBin dataset contains grayscale images obtained from malware binaries. Task 2 therefore has two goals: first, establish strong image-classification baselines; second, test whether a graph representation built from local image patches can provide a useful alternative representation.

The implementation was completed in a Kaggle notebook using PyTorch. Three CNN baselines were trained: SimpleCNN, ResNet18, and MobileNetV3Small. The proposed model is a custom PatchGCN in which a resized malware image is divided into an 8 x 8 grid. Each patch becomes a graph node, neighboring patches are connected, and statistical pixel features are used as node attributes. The models are evaluated on the same leakage-controlled split.

2. Task 2 Objectives

- Prepare the MaleBin images for leakage-aware, imbalanced multi-class classification.
- Train three representative CNN baselines and identify the strongest baseline.
- Convert every image into a patch graph with explicit nodes, edges, edge weights, and node features.
- Build and train a GCN using the patch graphs.
- Compare the first GCN result with the CNN baselines using class-balanced and efficiency metrics.
- Explain the proposed graph construction and GCN architecture in plain language.

3. Dataset Preparation and Data Integrity

Task 1 found 12,464 images across 39 malware families. Before Task 2 splitting, the notebook hashes image files with SHA-256, removes cross-class duplicate hash groups, and keeps only one copy of exact within-class duplicates. This is an important leakage-control step because identical images in both training and test sets would produce unrealistically high scores.

The deduplication stage reduced the usable set to 9,232 images. A minimum post-cleaning class size of 10 was then enforced, leaving 9,230 images across 35 classes. The cleaned set was stratified into 70% training, 15% validation, and 15% test partitions. The final split contains 6,461 training images, 1,384 validation images, and 1,385 test images.

Item	Value
Original MaleBin images	12,464
Original malware families	39
Cross-class duplicate hash groups detected	499
Usable images after duplicate handling	9,232
Final images after minimum-class filtering	9,230
Final classes used for Task 2	35
Training / Validation / Test	6,461 / 1,384 / 1,385
Split strategy	Stratified 70% / 15% / 15%

Table 3.1: Task 2 dataset preparation summary

3.1 Preprocessing and Imbalance Handling

CNN inputs are resized to 224 x 224 pixels, converted to tensors, and normalized from [0,1] to approximately [-1,1] using mean 0.5 and standard deviation 0.5. During CNN training, a small random translation is applied through RandomAffine. GCN inputs are resized to 128 x 128 pixels without CNN-style normalization because patch statistics are computed directly in the [0,1] intensity range.

Class imbalance is handled with a WeightedRandomSampler. Each training sample receives a weight inversely proportional to the number of training examples in its class. As a result, minority classes are sampled more frequently during training without altering the validation or test distributions.

4. CNN Baseline Models

Model	Main configuration	Purpose
SimpleCNN	3 convolution blocks (32, 64, 128 filters), batch normalization, ReLU, pooling, global average pooling, dropout 0.35	Lightweight baseline built specifically for grayscale malware images
ResNet18	Standard ResNet18 initialized from scratch; first convolution changed to 1 grayscale channel; final layer changed to 35 classes	Deeper residual baseline
MobileNetV3Small	MobileNetV3-Small initialized from scratch; grayscale first layer; final classifier changed to 35 classes	Compact mobile-oriented baseline

Table 4.1: CNN baseline configurations

All three baselines were initialized without pretrained ImageNet weights. This keeps the comparison focused on what can be learned directly from MaleBin, but it also means the transfer-learning advantage of pretrained CNNs was not tested in this Task 2 run.

5. Image-to-Graph Construction

Image-to-Graph Construction Used in Task 2

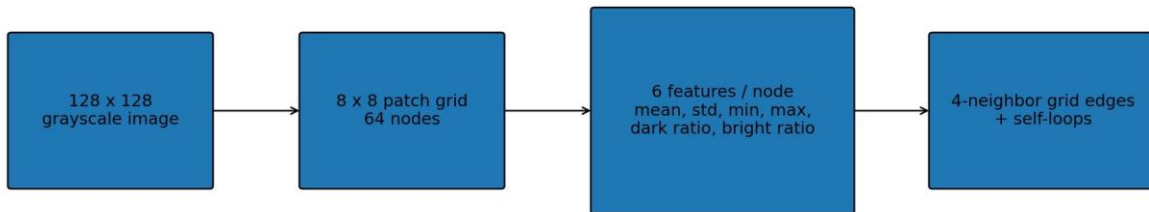


Figure 5.1: Patch-based image-to-graph construction

Each 128 x 128 grayscale image is divided into an 8 x 8 regular grid. Because each grid cell is 16 x 16 pixels, every image becomes a graph with 64 nodes. A node represents one local patch rather than an individual pixel.

For every patch, six statistical features are computed: mean intensity, standard deviation, minimum intensity, maximum intensity, dark-pixel ratio, and bright-pixel ratio. The dark ratio uses pixels below 50/255, while the bright ratio uses pixels above 205/255. These features summarize local texture and intensity without feeding all 256 pixels of a patch directly into the graph model.

Edges are based on spatial locality. A patch is connected to its immediate horizontal and vertical neighbors, and every node also receives a self-loop. The adjacency matrix is symmetrically normalized as $D^{-1/2} A D^{-1/2}$. Therefore, each graph-convolution step mixes a patch representation with information from nearby patches while controlling for node degree.

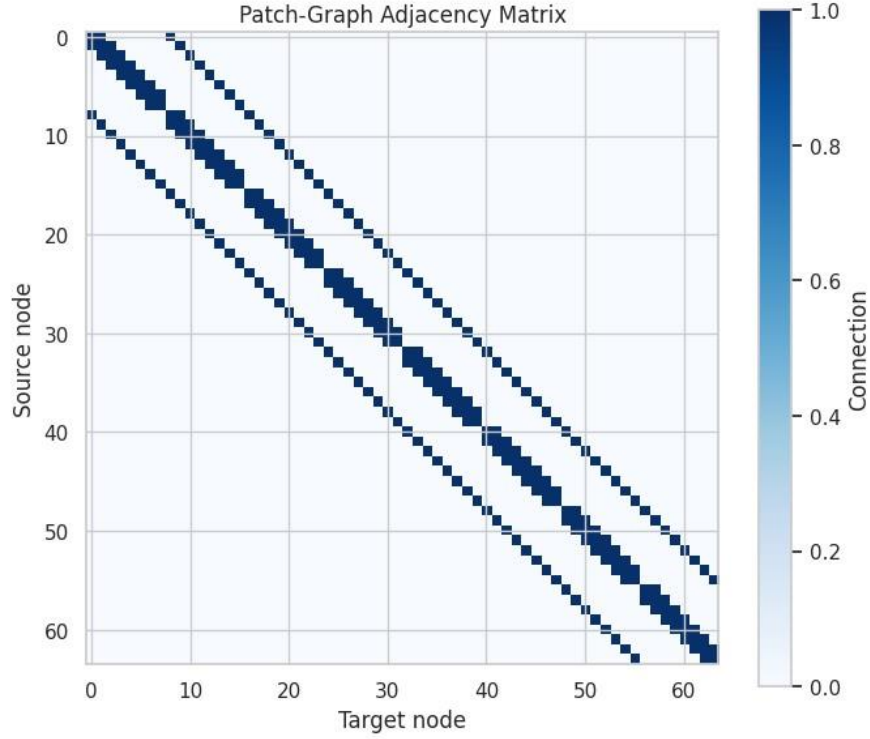
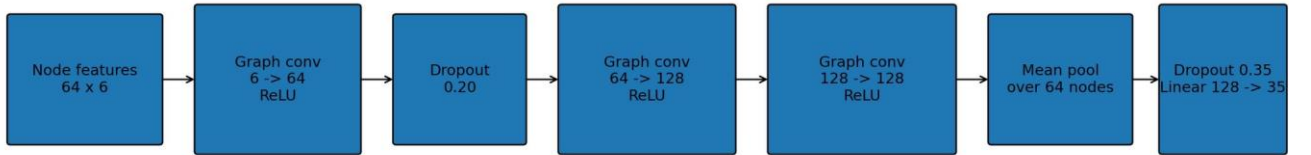


Figure 5.2: Adjacency matrix of the fixed 8 x 8 patch graph

This patch graph is a practical spatial abstraction for malware byte-images, which do not contain natural semantic regions such as those found in ordinary photographs or medical images. The graph preserves local spatial relationships between neighboring patches, but its nodes should be interpreted as image regions rather than semantically meaningful malware components.

6. Proposed PatchGCN Model

Proposed PatchGCN Architecture



Each graph layer performs normalized neighborhood aggregation using $\hat{A}XW$.

Figure 6.1: Proposed PatchGCN architecture

The PatchGCN contains three custom graph-convolution layers. The first maps the six patch features to 64 hidden features. The second maps 64 to 128 features, and the third keeps 128 features. ReLU activation follows every graph layer. A dropout rate of 0.20 is applied after the first graph layer. After the final layer, node representations are averaged across all 64 patches to obtain one 128-dimensional graph representation. A final dropout of 0.35 and a linear classifier produce the 35 malware-family logits.

In plain words, the model first summarizes each local patch, then repeatedly lets neighboring patches exchange information. The global mean pooling step compresses the 64 patch representations into one vector describing the whole malware image. This vector is then classified into one of the malware families.

7. Training and Evaluation Setup

Setting	Value
Random seed	42
Batch size	64
Maximum epochs	8
Optimizer	AdamW
Initial learning rate	0.001
Weight decay	0.0001
Learning-rate scheduler	ReduceLROnPlateau on validation macro-F1, factor 0.5, patience 1
Early stopping	Patience 3, best checkpoint selected by validation macro-F1
Gradient clipping	Maximum norm 1.0
Imbalance handling	WeightedRandomSampler on training set
Primary selection metric	Macro-F1

Table 7.1: Training configuration

The notebook reports accuracy, balanced accuracy, macro precision, macro recall, macro-F1, weighted precision, weighted recall, weighted F1, per-class classification reports, normalized confusion matrices, inference time per image, and saved model size. Macro-F1 is used as the main comparison metric because the cleaned dataset remains imbalanced.

The executed Task 2 notebook did not produce one-vs-rest ROC/AUC or precision-recall curves. These metrics are therefore not reported in this document; all numerical results presented here come directly from the completed Task 2 experiments.

8. Experimental Results

Model	Accuracy	Bal. Acc.	Macro P	Macro R	Macro F1	Weighted F1	ms/image	Size MB
ResNet18	0.8664	0.8811	0.8576	0.8811	0.8615	0.8648	1.8352	42.7546
SimpleCNN	0.6852	0.6757	0.6735	0.6757	0.6475	0.6661	1.4415	0.3814
MobileNetV3 Small	0.6469	0.6856	0.6894	0.6856	0.6500	0.6591	1.4972	6.0531
PatchGCN	0.4433	0.5080	0.3864	0.5080	0.3771	0.3887	1.7656	0.1326

Table 8.1: Test-set comparison of CNN baselines and PatchGCN

ResNet18 is the strongest Task 2 model, achieving 86.64% test accuracy and a macro-F1 of 0.8615. It also obtains the highest balanced accuracy and weighted F1. The other two CNN baselines are substantially lower, with macro-F1 values of 0.6475 for SimpleCNN and 0.6500 for MobileNetV3Small.

The proposed PatchGCN reaches 44.33% accuracy and a macro-F1 of 0.3771. Therefore, the first graph model does not outperform the CNN baselines. The result indicates that compressing each image patch into only six statistical features removes fine-grained texture information that is useful for malware-family classification.

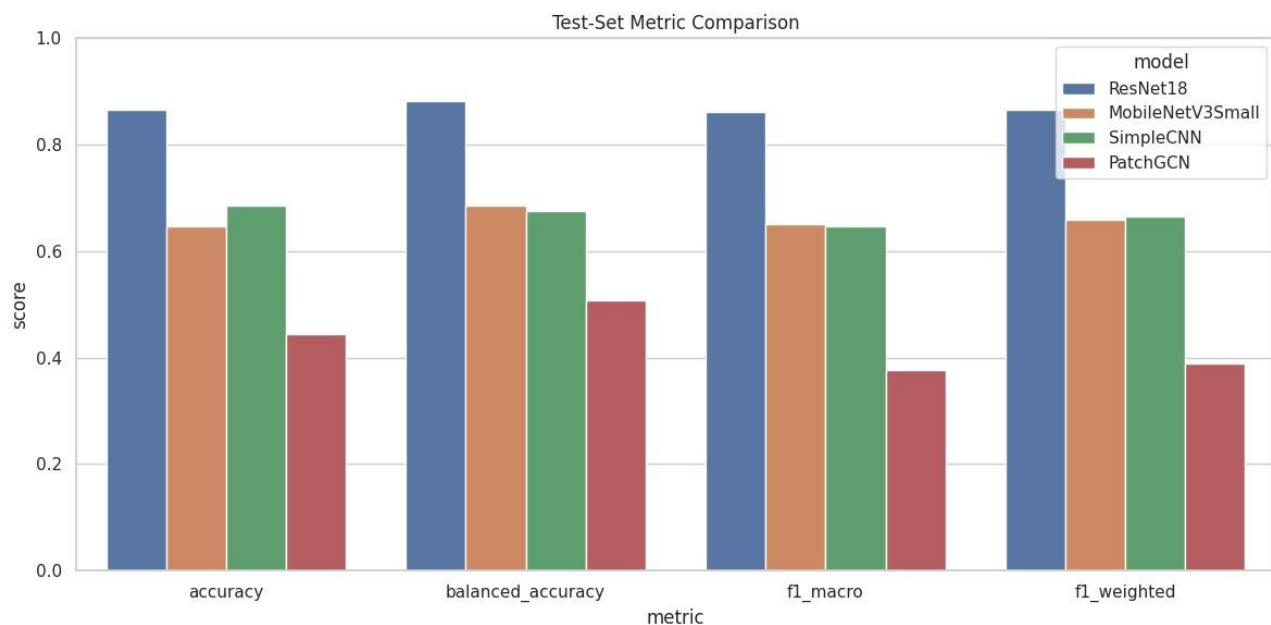


Figure 8.1: Test-set metric comparison from the Task 2 notebook 8.1

Training Behavior

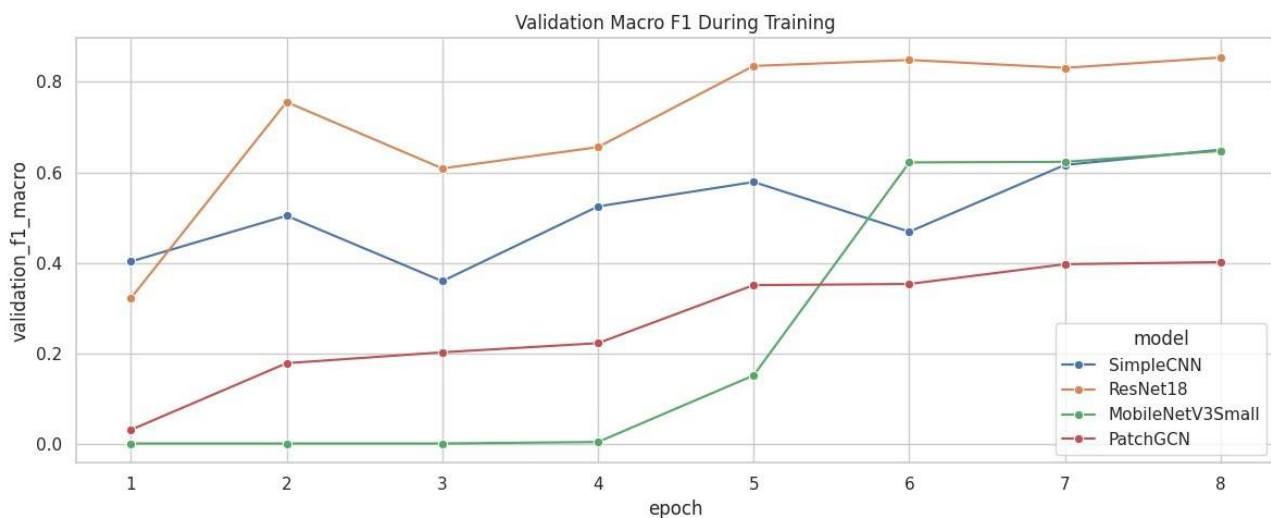


Figure 8.2: Validation macro-F1 across training epochs

ResNet18 improves quickly after the first epoch and finishes with the highest validation macro-F1. SimpleCNN improves more gradually. MobileNetV3Small remains near zero during the early epochs before learning substantially in later epochs. PatchGCN improves across the run but plateaus far below the CNN models. The curve suggests that the PatchGCN is learning real class structure, but the current graph features are not expressive enough for competitive classification.

8.2 Class-Level Behavior

The normalized confusion matrices show that ResNet18 produces a strong diagonal across most malware families, while PatchGCN has substantially more off-diagonal confusion. PatchGCN performs particularly well on some visually distinctive families such as Allapple.L (F1 = 0.980), Lolyda.AA3 (0.973), and Dialplatform.B

(0.931). However, several families receive zero recall, including CoinMinerXMRig, Glupteba, Remcos, Swizzor.genII, Trickbot, Wintrim.BX, and Zeus. This explains the large gap between its accuracy and its macro-F1.

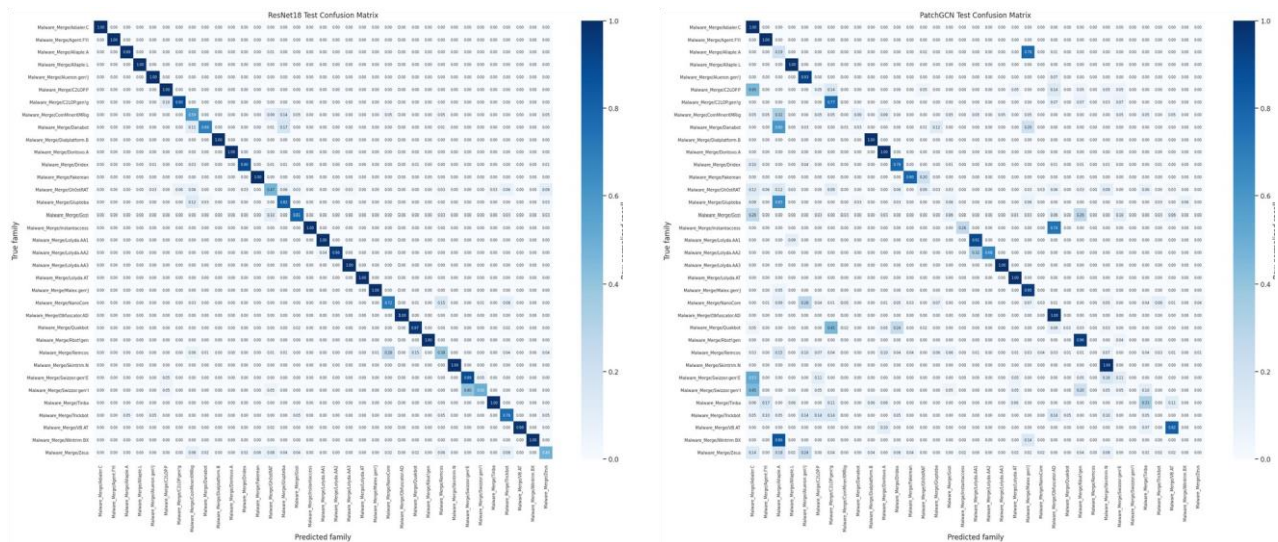


Figure 8.3: Normalized confusion matrices for ResNet18 (left) and PatchGCN (right)

8.3 Efficiency

PatchGCN is the smallest saved model at about 0.13 MB, compared with 0.38 MB for SimpleCNN, 6.05 MB for MobileNetV3Small, and 42.75 MB for ResNet18. Its measured inference time is about 1.77 ms per image, which is close to the CNN baselines. Thus, the graph model is compact, but the current accuracy loss is too large to justify choosing it over the stronger CNN baseline.

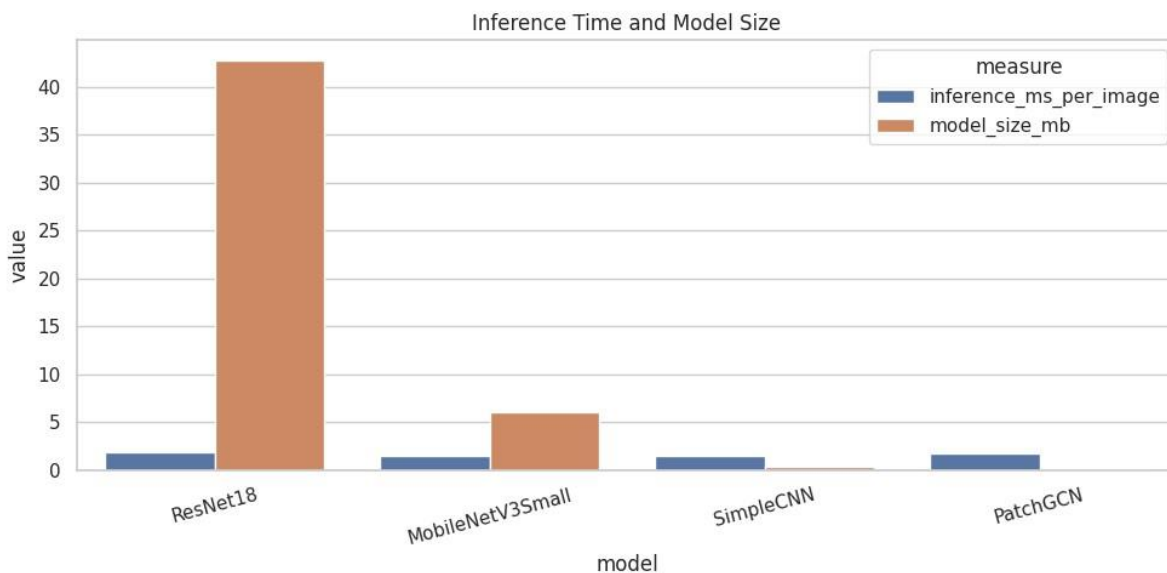


Figure 8.4: Inference time and saved model size

9. Discussion

The main Task 2 finding is that ResNet18 is the baseline to beat in later work. Its macro-F1 of 0.8615 is more than twice the first PatchGCN macro-F1. The result is consistent with the nature of MaleBin: malware

byteimages often contain repeated textures and fine visual patterns that standard convolutions can capture directly, whereas the current graph compresses each 16 x 16 patch into only six summary values.

The graph design is nevertheless valid as a first Track 2 proposal because its nodes, edges, edge weights, node features, and pooling operation are explicitly defined. The graph is also constructed without using test labels or test-derived statistics. However, the first result shows that the representation must be improved before a graph model can compete with ResNet18.

For Task 3, the most defensible improvements are to increase node expressiveness and test alternative graph constructions. Examples include smaller patches, CNN feature vectors as node features, similarity-weighted edges, GraphSAGE or GAT variants, additional GCN depth/width choices, stronger regularization, and an ablation comparing CNN versus CNN+GCN. The final comparison should retain the same held-out test set and use macro-F1 as a primary metric.

10. Limitations of the Current Task 2 Run

- The first PatchGCN uses only six handcrafted statistical features per patch; detailed byte-image texture is heavily compressed.
- The graph topology is fixed for every image and uses only spatial neighborhood, not content similarity or learned edge weights.
- ResNet18 and MobileNetV3Small were trained from scratch; pretrained transfer-learning baselines were not evaluated.
- The executed notebook does not yet include multiclass one-vs-rest ROC/AUC and precision-recall curves required by the course reporting rules.
- The experiments in this phase were executed in a single Kaggle notebook containing both the CNN baselines and the proposed graph model.

11. Conclusion

Task 2 successfully established three CNN baselines and implemented a complete patch-based image-to-graph pipeline with a custom GCN. After duplicate handling and class filtering, 9,230 images from 35 malware families were used. ResNet18 is the best baseline with 86.64% accuracy and 0.8615 macro-F1. The proposed PatchGCN is compact but achieves only 44.33% accuracy and 0.3771 macro-F1, so the first graph representation does not yet improve on conventional CNNs.

The result provides a clear direction for the next phase: retain ResNet18 as the reference baseline, improve the graph representation, perform controlled ablation studies, and evaluate whether richer node features or alternative graph constructions can reduce the performance gap.

References

- [1] "MaleBin Malware Binary Greyscale Images," Kaggle dataset.
<https://www.kaggle.com/datasets/tashiee/malebin-malware-binary-greyscale-images>
- [2] T. N. Kipf and M. Welling, "Semi-Supervised Classification with Graph Convolutional Networks," ICLR, 2017, arXiv:1609.02907.
- [3] L. Nataraj et al., "Malware Images: Visualization and Automatic Classification," VizSec, 2011.